

 inclusionAI / **Ling-3.0-flash-fp8** 

 like 32

Follow  inclusionAI 2.6k

 Text Generation  Safetensors `bailing_hybrid` `conversational` `custom_code` `fp8`

 License: mit

⋮

→ Copy to bucket **NEW**

 **Model card**

Files

 xet

 Community 7



 [Hugging Face](#) |  [ModelScope](#) |  [OpenRouter](#)

Introduction

We're introducing Ling-3.0-flash, our next-generation native hybrid reasoning model. Operating with **124B** total and **5.1B** active parameters (~12.4% and ~8.1% of our previous 1T-class flagship Ring-2.6-1T), Ling-3.0-flash matches or outperforms its predecessor across key benchmarks.

Key highlights of the model are summarized below:

- **Native Hybrid-Linear Architecture:** Ling-3.0 adopts a native hybrid linear attention architecture from the very start of pretraining

Downloads last month
2,673



 **Safetensors** 

Model size 127B params

Tensor type F32 · BF16 · F8_E4M3

 Chat template

 Files info

 **Inference Providers** **NEW**

 Text Generation

This model isn't deployed by any Inference Provider.

 Ask for provider support

 **Collection including** inclusionAI/...

Ling 3.0  Collection

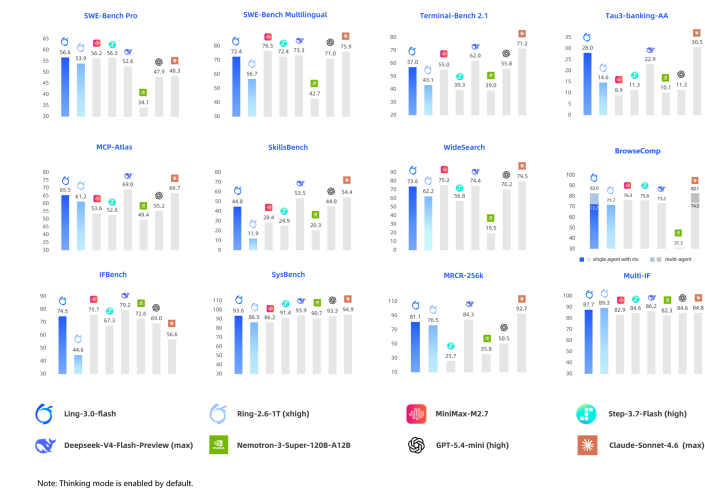
Lin... • 7 items • Updated 3 d... •  12

(5:1 alternating stacking of Kimi Delta Attention (KDA) and MLA), upgraded with KDA fine-grained diagonal gating and 1/64 sparse MoE. With 124B total parameters and 5.1B activated parameters, it achieves a synergistic leap in long-context efficiency and computational cost.

- **Remarkable Efficiency & Performance:**

Engineered for speed, compute efficiency, and production deployment, Ling-3.0-flash delivers class-defying performance against both larger SOTA competitors and previous-generation flagships. Activating only 5.1B parameters per token, it provides impressive reasoning, instruction following, and long-context capabilities to empower complex agentic workflows in production environments.

- **Comprehensive Agentic Evolution:** Tailored for real-world productivity workflows, the model incorporates 10,000+ interactive training environments to achieve end-to-end closed-loop execution across Coding, General, and Deep Research Agent tasks. It natively integrates the SGLang HiCache + Mooncake hierarchical caching architecture (featuring physical dual-pools and a cluster-shared L3 cache), eliminating redundant recomputation during long-horizon interactions and reducing Time to First Token (TTFT) by 60% to over 80% in long-input scenarios.

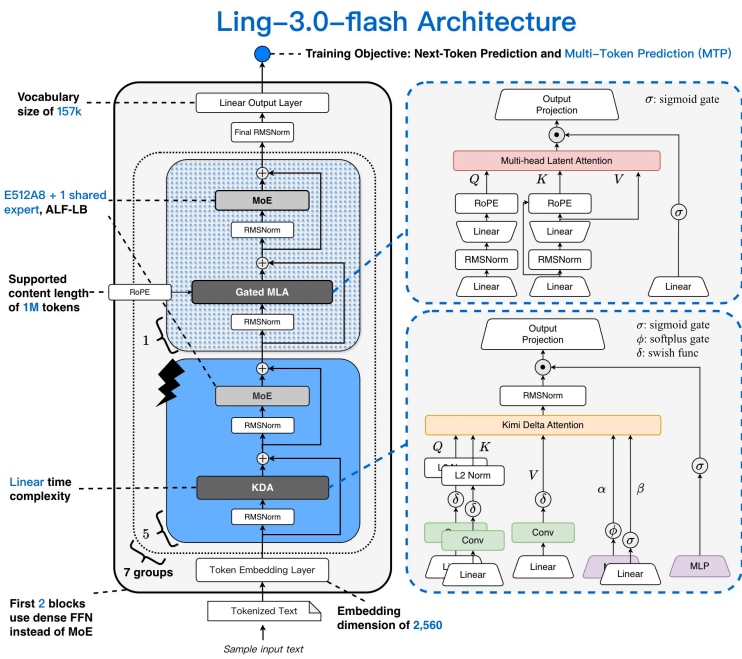


Model Overview

The model summary information and architecture diagram are as follows:

Architect ure	Hybrid -linear MoE
Parameter Scale	Total 124B, Activated 5.1B
Transformer Layers	35 KDA + 7 Gated MLA (5:1)
Number of Dense Layers	2
Number of Routed Experts	512
Number of Shared Experts	1
Number of Activated Experts	8
Attention Heads	32
Hidden Size	2560
Expert Intermediate Size	768
Dense Intermediate Size	6144
Vocabulary Size	157184

Architecture	Hybrid-linear MoE
Context Training Schedule	8K -> 32K -> 256K



Evaluation

We have conducted a comprehensive evaluation of Ling-3.0-flash across multiple authoritative benchmarks. **Ling-3.0-flash** performs strongly on representative code/agent benchmarks such as **SWE-Bench Pro**, **SWE-Bench Multilingual**, **Tau3-banking-AA**, **MCP-Atlas** and **SkillsBench**, etc. In practice, Ling-3.0-flash delivers a strong user experience across frameworks including **Claude Code**, **Kilo Code**, **Qwen Code**, **Hermes Agent**, and **OpenClaw**, etc. Beyond agentic tasks, Ling-3.0-flash also delivers strong performance across **general knowledge**, **mathematical reasoning**, **instruction following**, and **long-context understanding**.

	Ling-3.0-flash	Ring-2.6-1T (xhigh)	MiniMax-M2.7	Step-3.7-Flash (high)	Deepseek-V4-Flash-Preview (max)	Nemotron-3-Super-120B-A12B	GPT-5.4-mini (high)	Claude-Sonnet-4.6 (max)
Size	124B-A5.1B	1T-A63B	230B-A10B	198B-A11B	284B-A13B	120B-A12B	-	-
Coding Agent								
SWE-Bench Pro	56.6	53.9	56.2	56.3	52.6	34.1	47.9	48.3
SWE-Bench Multilingual	72.4	56.7	76.5	72.4	73.3	42.7	71.0	75.9
Terminal-Bench 2.1	57.0	43.1	55.0	39.3	62.0	39.0	55.8	71.2
ArtifactsBench	77.0	65.1	55.8	59.2	64.0	51.6	66.8	68.7
MiniAppBench	25.3	19.2	20.7	28.0	14.8	5.8	58.8	46.3
AntSWEBench	52.2	46.4	-	-	48.6	-	-	-
General Agent								
Tau3-banking-AA	28.0	14.6	8.9	11.3	22.9	10.1	11.3	30.5
MCP-Atlas	65.5	61.2	53.6	52.6	69.0	49.4	55.2	66.7
SkillsBench	44.8	11.9	28.4	24.9	53.5	20.3	44.8	54.4
BFCL-v4	73.0	64.8	63.6	65.4	59.5	60.6	68.3	73.1
GDPVal v2-AA	1107	920	1159	1017	1189	699	-	1377
Search Agent								
WideSearch	73.6	62.2	75.2	56.8	74.4	19.5	70.2	79.5
BrowseComp	72.2 (w/ ctx) 82.0 (MA)	71.7	76.3	75.8	73.2	31.3	-	74.0 (w/ ctx) 82.1 (MA)
Draco	70.4	-	66.8	-	71.3	-	61.3	75.8
Instruction Following								
IFBench	74.5	44.6	75.7	67.3	79.2	72.6	69.0	56.6
SysBench	93.6	86.5	86.2	91.4	93.9	90.7	93.3	94.9
LIFEBench	77.3	72.5	66.9	71.3	74.1	60.2	69.2	71.8
Reasoning								
AIME26	93.2	95.8	94.2	95.0	96.5	91.7	92.9	94.4
HMMT-Feb26	87.0	93.5	71.9	87.9	94.8	84.9	83.9	85.6
IMO-AnswerBench	83.7	86.1	66.9	-	87.0	77.0	74.5	82.1
HLE	22.7	18.3	28.1	19.9	34.8	18.3	20.6	30.0
LiveCodeBench (2408-2505)	82.8	87.0	75.6	78.1	91.6	78.7	80.8	83.7
Long Context & Multi-Turn Dialogue								
MRRCR_128K	90.8	90.1	27.7	39.2	88.5	40.8	56.1	92.5
MRRCR_256k	81.1	76.5	-	25.7	84.3	35.8	50.5	92.7
AA-LCR	65.1	64.3	68.7	63.7	63.0	58.3	63.4	70.7
Multi-IF	87.7	89.3	82.9	84.6	86.2	82.3	84.6	84.8

- Thinking mode is enabled by default. Unless otherwise specified, the default parameters for Ling-3.0-flash are as follows: `temperature=0.6`, `top_p=0.95`, `top_k=20`.
- SWE-Bench Series: Evaluated using OpenHands as the agent harness with tailored prompts. Decoding uses `temperature=0.6`, `top_p=0.95`, `max_new_tokens=32K`, with a 256K context window.
- Terminal-Bench 2.1: Evaluated under the Artificial Analysis (AA) protocol using the default Terminus 2 harness, a unified 2-hour timeout, the provided JSON parser in preserve-thinking mode, and 3 runs per task (mean). Decoding uses `temperature=0.6`, `top_p=1.0`, `max_new_tokens=32K`, with a 256K context window.

- *MiniAppBench: A 500-task coding benchmark evaluating whether models can turn a single user request into complete, usable interactive HTML apps in real-world application-generation scenarios. Evaluated with `temperature=1.0`, `top_p=1.0`, `max_tokens=128K`.*
- *AntSWEBench: AntSWEBench is an internally used software engineering benchmark that covers mainstream programming languages such as Java, JavaScript, and Python, including various development scenarios like new feature, bug fix, and code refactoring.*
- *Tau3-banking-AA: Aligned with the AA leaderboard, utilizing GPT-5.4-mini (medium reasoning) for both the user simulator and the natural-language assertion judge.*
- *MCP-Atlas: Evaluated on the 500-task public set using the official v1 harness with a 20-turn limit and Gemini-2.5-Pro as the claim-coverage judger.*
- *SkillsBench: Evaluated via kilo-code on 87 tasks (excluding external API-dependent tasks), averaged over 3 runs.*
- *GDPval v2-AA: Evaluated on the public 220-task benchmark using the official Stirrup harness, with a 250-turn limit and a 5-hour timeout.*
- *Search-agent: For all search-agent tasks, evaluations are performed using an internal harness. The basic ReAct paradigm is adopted for single-agent evaluation, while a multi-agent setup is employed for BrowseComp. The reported metric is the average pass@1.*
 - *WideSearch: Evaluated using the official prompt and the official judge model GPT-4.1 on the corrected version of the dataset.*
 - *Draco: Scored based on official rubrics per question, with the final score calculated as the average across all questions using Claude Opus 4.6 as the scoring model.*

- *BrowseComp (Single-Agent): Evaluated using a resume strategy for context management: once the context reaches a 64K-token threshold, the trajectory is summarized, the original history is discarded, and execution is resumed from the summary.*
- *BrowseComp (Multi-Agent): Evaluated using an internal multi-agent search harness based on SearchSwarm/Tongyi DeepResearch, configured with temperature=0.85, top_p=0.95, max_tokens=8K, and main/sub-agent context windows of 128K and 64K, respectively.*

Quantized Models

We evaluate the quantized models using several datasets. The FP8 quantized model is applied via the blockwise quantization, and INT4 and FP4 models are applied via groupwise quantization with routed experts weights.

dataset	BF16	FP8	INT4	FP4
GPQA-diamond	84.97	84.00	83.65	82.42
IFBench	74.49	73.40	72.20	72.33
SciCode	41.24	40.37	39.35	39.79
ArcPrize	68.75	67.18	67.56	64.16

Quickstart

SGLang

The hardware- and recipe-specific launch matrix (BF16/FP8 × Low-Latency / High-Throughput / HiCache + Mooncake), with a live command generator and verified configurations, lives in the SGLang cookbook:

Cookbook:

<https://docs.sglang.io/cookbook/autoregressive/InclusionAI/Ling-3.0-flash>

Install SGLang

Use the pre-built image that tracks the Ling-3.0 runtime:

```
docker pull lmsysorg/sglang:dev-Ling-3.0-flash
```

Run Inference

Recommended low-latency recipe for the FP8 checkpoint (built-in MTP / NEXTN, 256K YaRN context) on 4× 141GB-class GPUs (H20-3e) or 4-GPU Blackwell nodes. The checkpoint uses blockwise (128×128) E4M3 expert weights, so the FP8 recipes pair TP with expert parallelism; SGLang detects the quantization format from the checkpoint's `quantization_config`, so no explicit quantization flag is needed:

```
docker run --rm --gpus all --ipc=host --shm-size=1GB \
  -p 30000:30000 \
  -e HF_TOKEN=<your-hf-token> \
  lmsysorg/sglang:dev-Ling-3.0-flash \
  env SGLANG_ALLOW_OVERWRITE_LONGER_CONTEXT_LENGTH=1 \
  python3 -m sglang.launch_server \
    --model-path inclusionAI/Ling-3.0-flash-1.0 \
    --tp 4 \
```

```
--ep-size 4 \  
--context-length 262144 \  
--speculative-algorithm NEXTN \  
--mem-fraction-static 0.8 \  
--host 0.0.0.0 \  
--port 30000
```

On H100 / H800 use `--tp 8 --ep-size 8` with the same flags; see the cookbook cell for your hardware.

Client

Thinking is enabled by default by both the chat template and the `ling3` reasoning parser. Disable it per request with `"chat_template_kwargs": {"enable_thinking": false}`. We recommend the sampling parameters `temperature=0.6`, `top_p=0.95`, and `top_k=20`.

```
curl -s http://localhost:30000/v1/chat/completions \\  
-H "Content-Type: application/json" \  
-d '{"model": "inclusionAI/Ling-3.0-flash-8bit", \  
  "messages": [{"role": "user", "content": "Hello!"}], \  
  "stream": true, \  
  "temperature": 0.6, \  
  "top_k": 20, \  
  "top_p": 0.95 \  
'
```

For `--reasoning-parser ling3` / `--tool-call-parser ling3`, the HiCache + Mooncake L3 setup, and GSM8K / `bench_serving` reproduction commands, see the cookbook page linked above.

vLLM

Install our vLLM

```
pip install uv
```

```
uv venv ~/my_ling_env
```

```
source ~/my_ling_env/bin/activate
```

```
git clone -b ling_3_0 https://github.com/inc.
```

```
cd vllm-ling-v3
```

```
VLLM_USE_PRECOMPILED=1 uv pip install --editi
```

Run Inference

Here is the example to run Ling-3.0-flash with 2 GPUs, where the server port is \${PORT}:

Server

Since the model is trained with MTP, we recommend enabling MTP during inference (i.e., --speculative-config) for lower latency.

```
vllm serve "$MODEL_PATH" \  
  --port "$PORT" \  
  --trust-remote-code \  
  --served-model-name auto \  
  --tensor-parallel-size 2 \  
  --gpu-memory-utilization 0.85 \  
  --enable-prefix-caching \  
  --mamba-cache-mode align \  
  --enable-auto-tool-choice \  
  --tool-call-parser ling3 \  
  --reasoning-parser ling3 \  
  --speculative-config '{"method":"mtp","n"
```

Client

We recommend using the sampling parameters
temperature=0.6, top_p=0.95, and top_k=20, and
enabling enable_thinking for better performance.

 System theme

[TOS](#)

[Privacy](#)

[About](#)

[Careers](#)



[Models](#)

[Datasets](#)

[Spaces](#)

[Pricing](#)

[Docs](#)

```
curl -s http://${MASTER_IP}:${PORT}/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{"model": "auto",
    "messages": [{"role": "user", "content": "What is the meaning of life?"}],
    "chat_template_kwargs": {"enable_thinking": true},
    "stream": true,
    "temperature": 0.6,
    "top_k": 20,
    "top_p": 0.95
}'
```